

Linear Regression

Mamta Bhattarai

June 29, 2026

Linear Regression is a supervised learning algorithm used to model the relationship between input variables and a continuous output variable.

Applications:

- House price prediction
- Sales forecasting
- Stock market analysis
- Medical data analysis

Simple Linear Regression

For one feature:

$$\hat{y} = mx + b$$

where

- m = slope
- b = intercept
- $\hat{y}$ = predicted value

$$b = \bar{y} - m\bar{x}$$

Multiple Linear Regression

For multiple features:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Matrix form:

$$\hat{y} = X\theta$$

where

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} \\ 1 & x_{21} & x_{22} \\ \vdots & \vdots & \vdots \end{bmatrix}$$

Cost Function

The objective is to minimize prediction error.

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

where

$$\hat{y}^{(i)} = \theta^T x^{(i)}$$

Goal:

$$\min_{\theta} J(\theta)$$

Optimization using Gradient Descent

Parameter update rule:

$$\theta_j = \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

where

- α = learning rate
- $\frac{\partial J}{\partial \theta_j}$ = gradient

Gradient:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

Gradient Descent Algorithm

Input:

- Training data (X, y)
- Learning rate α
- Number of iterations T

Algorithm

- 1 Initialize θ
- 2 Compute predictions:

$$\hat{y} = X\theta$$

- 3 Compute cost:

$$J(\theta)$$

- 4 Compute gradients
- 5 Update parameters

$$\theta = \theta - \alpha \nabla J$$

- 6 Repeat until convergence

Two Feature Problem

Suppose we want to predict house price using:

- x_1 = House Size
- x_2 = Number of Rooms

Dataset:

Size (x_1)	Rooms (x_2)	Price (y)
1000	2	200
1500	3	300
2000	4	400
2500	4	450

Data Matrix

Add a bias term:

$$X = \begin{bmatrix} 1 & 1000 & 2 \\ 1 & 1500 & 3 \\ 1 & 2000 & 4 \\ 1 & 2500 & 4 \end{bmatrix}$$

Target vector:

$$y = \begin{bmatrix} 200 \\ 300 \\ 400 \\ 450 \end{bmatrix}$$

Parameter vector:

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}$$

Prediction Equation

The regression model is

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Assume after training

$$\theta = \begin{bmatrix} 20 \\ 0.15 \\ 15 \end{bmatrix}$$

Then

$$\hat{y} = 20 + 0.15x_1 + 15x_2$$

Prediction Example

Predict price for

$$x_1 = 1800$$

$$x_2 = 3$$

Using

$$\hat{y} = 20 + 0.15x_1 + 15x_2$$

$$\hat{y} = 20 + 0.15(1800) + 15(3)$$

$$\hat{y} = 20 + 270 + 45$$

$$\hat{y} = 335$$

Normal Equation

Instead of Gradient Descent, parameters can be computed directly:

$$\theta = (X^T X)^{-1} X^T y$$

Advantages:

- No learning rate
- Exact solution
- Fast for small datasets

Advantages

- Simple and interpretable
- Fast training
- Works well for linear relationships
- Easy to implement

Limitations

- Assumes linear relationship
- Sensitive to outliers
- Multicollinearity affects performance
- Poor for highly nonlinear data

Summary

- Linear Regression predicts continuous values.
- Model:

$$\hat{y} = X\theta$$

- Objective:

$$\min J(\theta)$$

- Optimization:
 - Gradient Descent
 - Normal Equation
- Multiple features can be handled naturally.

Derivation of the Gradient

Recall the cost function:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

Using

$$\hat{y}^{(i)} = \theta^T x^{(i)}$$

we obtain

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2$$

Goal:

$$\min_{\theta} J(\theta)$$

Partial Derivative with Respect to θ_j

Differentiate the cost function:

$$\frac{\partial J}{\partial \theta_j} = \frac{\partial}{\partial \theta_j} \left[\frac{1}{2m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2 \right]$$

Applying the chain rule:

$$= \frac{1}{2m} \sum_{i=1}^m 2(\theta^T x^{(i)} - y^{(i)}) \frac{\partial}{\partial \theta_j} (\theta^T x^{(i)} - y^{(i)})$$

Since

$$\frac{\partial}{\partial \theta_j} (\theta^T x^{(i)}) = x_j^{(i)}$$

we get

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

Gradient Descent Update Rule

Using

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

the parameter update becomes

$$\theta_j = \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

Therefore,

$$\theta_j = \theta_j - \alpha \left(\frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} \right)$$

for

$$j = 0, 1, \dots, n$$

Matrix Form of the Gradient

Cost function:

$$J(\theta) = \frac{1}{2m}(X\theta - y)^T(X\theta - y)$$

The gradient is

$$\nabla J(\theta) = \frac{1}{m}X^T(X\theta - y)$$

Hence the vectorized update rule becomes

$$\theta = \theta - \alpha \frac{1}{m}X^T(X\theta - y)$$

This form is commonly used in MATLAB and Python implementations.

Given dataset:

i	x_1	x_2	y
1	1	1	6
2	1	2	8
3	2	1	7

Model:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

—

$$\theta_0 = 0, \quad \theta_1 = 0, \quad \theta_2 = 0$$

$$\alpha = 0.1, \quad m = 3$$

—

Since $\theta = 0$:

$$\hat{y}^{(i)} = 0$$

Error:

$$\hat{y} - y = \begin{bmatrix} -6 \\ -8 \\ -7 \end{bmatrix}$$

$$\theta_0 = 0 - 0.1 \cdot \frac{-21}{3} = 0.7$$

$$\sum (\hat{y} - y)x_1 = -28$$

$$\theta_1 = 0 - 0.1 \cdot \frac{-28}{3} = 0.933$$

$$\sum (\hat{y} - y)x_2 = -29$$

$$\theta_2 = 0 - 0.1 \cdot \frac{-29}{3} = 0.967$$

$$\theta_0 = 0.7, \quad \theta_1 = 0.933, \quad \theta_2 = 0.967$$

$$\theta_0 = 1.075, \quad \theta_1 = 1.442, \quad \theta_2 = 1.409$$

$$X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad y = \begin{bmatrix} 6 \\ 8 \\ 7 \end{bmatrix}$$

$$\theta = (X^T X)^{-1} X^T y$$

In linear regression, we aim to minimize the cost function:

$$J(\theta) = \frac{1}{2m} (X\theta - y)^T (X\theta - y)$$

where:

- $X \in \mathbb{R}^{m \times n}$ is the data matrix

- $y \in \mathbb{R}^{m \times 1}$ is the output vector
 - $\theta \in \mathbb{R}^{n \times 1}$ is the parameter vector
-

$$J(\theta) = \frac{1}{2m} \left(\theta^T X^T X \theta - 2y^T X \theta + y^T y \right)$$

Using standard matrix derivatives:

$$\nabla_{\theta} J(\theta) = \frac{1}{2m} \left(2X^T X \theta - 2X^T y \right)$$

Simplifying:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \left(X^T X \theta - X^T y \right)$$

At the optimal point:

$$\nabla_{\theta} J(\theta) = 0$$

$$X^T X \theta = X^T y$$

Assuming $X^T X$ is invertible:

$$\theta = (X^T X)^{-1} X^T y$$

$$\theta = (X^T X)^{-1} X^T y$$

This is known as the **Normal Equation** or **Closed-Form Solution** of Linear Regression.

$$X^T X = \begin{bmatrix} 3 & 4 & 4 \\ 4 & 6 & 5 \\ 4 & 5 & 6 \end{bmatrix}$$

$$X^T y = \begin{bmatrix} 21 \\ 28 \\ 29 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 4 & 4 \\ 4 & 6 & 5 \\ 4 & 5 & 6 \end{bmatrix} \theta = \begin{bmatrix} 21 \\ 28 \\ 29 \end{bmatrix}$$

Solving gives:

$$\theta_0 = 1, \quad \theta_1 = 2, \quad \theta_2 = 1$$

Method	Parameters	Nature
Gradient Descent	(1.075, 1.442, 1.409)	Approximate (iterative)
Normal Equation	(1, 2, 1)	Exact solution

- Gradient Descent improves parameters step-by-step.
- Normal Equation gives direct closed-form solution.
- Both converge to similar values for proper learning rate and iterations.